

AI Query Optimizer

Beginner Project Guide

Machine Learning * LLM * PostgreSQL * Streamlit

Understand - Explain - Defend

Table of Contents

1.	What is this Project?	The core idea in plain English
2.	Real-World Problem & Why It Matters	Why slow queries happen
3.	Objective & Scope	What the app does and does not do
4.	Technology Stack	Every tool explained simply
5.	Why These Technologies?	Choices and trade-offs
6.	System Architecture	How all pieces connect
7.	Feature-by-Feature Walkthrough	Every screen and function
8.	End-to-End Workflow	Step-by-step from input to output
9.	Data, Model & Files	What files do what
10.	Security & Responsible Design	Why it is built safely
11.	Deployment Guide	Local and Streamlit Cloud
12.	Challenges & How They Were Solved	Engineering decisions
13.	Limitations - Be Honest	What the project cannot do
14.	Future Improvements	What could be added next
15.	How to Explain in an Interview	30-sec and 90-sec answers
16.	Counter Questions & Answers	25 tough questions
17.	Glossary	All terms explained simply

1. What Is This Project?

The core idea in plain English

Imagine you run a shop with a million product records. You ask the database: "Show me all red shoes under Rs. 2000." The database does not just blindly read every row. It first makes a **plan** - a recipe - for how to find the answer as efficiently as possible.

This project is a **dashboard** that lets you look inside that plan, understand it, and get guidance on whether anything looks suspicious or slow. Think of it as an X-ray machine for your SQL query - it shows you what is happening under the hood.

Key Point

One-line summary: AI Query Optimizer reads PostgreSQL execution plans, compares row estimates using a trained ML model, and gives beginners clear, safe guidance - all in a visual web app.

What is a Query Plan?

When you send SQL to PostgreSQL, the database engine does not just run it immediately. It first plans how to answer it. The plan includes steps (called **Plan Nodes**) such as scanning a table, filtering rows, joining tables, and sorting.

- Scan this table (check every row or use an index shortcut).
- Filter rows matching the WHERE condition.
- Join results from two or more tables.
- Sort the final result.
- Return a limited number of rows (LIMIT).

2. Real-World Problem and Why It Matters

Why slow queries happen

The Problem: Estimate Errors

PostgreSQL uses statistics to estimate how many rows each plan node will return. When these estimates are inaccurate (e.g. data grown significantly, skewed data, or combined filters), PostgreSQL can pick a bad plan.

Situation	Bad Estimate Effect	Real Impact
Join estimated 10 rows, actual 100,000	PostgreSQL picks Nested Loop	Millions of comparisons instead of thousands
Filter estimated 50%, actual 1%	No index is used	Full table scan on 10 million rows
Sort estimated small, actual huge	Too little memory allocated	Disk spill, very slow sort

3. Objective and Scope

What the app does and does not do

This project IS	This project IS NOT
An explainability and learning assistant	A production-grade autonomous optimizer
A visual comparison of ML vs PostgreSQL estimates	A replacement for PostgreSQL query planner
A safe recommendation system	A system that automatically creates indexes
A beginner-friendly starting point for diagnostics	A universal model for all databases
A Streamlit web app deployable on the cloud	An enterprise SaaS product

4. Technology Stack

Every tool explained simply

Python 3

What: The core programming language used.

Why: Huge ecosystem for ML, data manipulation, and Web apps.

Streamlit

What: Interactive web application framework.

Why: Pure Python UI creation without frontend code.

PostgreSQL

What: Relational database engine analyzed.

Why: Provides structured EXPLAIN JSON execution plans.

XGBoost

What: Machine learning model predicting row counts.

Why: Gradient boosting excels on structured tabular plan features.

scikit-learn & joblib

What: ML preprocessing and serialization.

Why: Prepares data and saves trained models to disk (.pkl).

Pandas & NumPy

What: DataFrames and numeric processing.

Why: Calculates Q-error and parses plan nodes efficiently.

Plotly

What: Interactive data visualization.

Why: Renders hoverable charts directly in Streamlit.

Google Gemini API

What: LLM for English-to-SQL & Query Doctor.

Why: Provides plain-English interpretations and explanations.

Neon & Streamlit Cloud

What: Cloud database and hosting.

Why: Enables public deployment without local machine dependencies.

5. System Architecture

How all pieces connect

Layer	What it does	Technologies
Presentation	User interface & controls	Streamlit, Plotly
Validation	Read-only SELECT filter & SQL extraction	Python Regex
Database	Execute EXPLAIN (ANALYZE, FORMAT JSON)	psycopg2, PostgreSQL
ML Engine	Cardinality prediction via XGBoost	XGBoost, joblib
AI Doctor	Plain-English plan diagnosis	Google Gemini API
Logging	Session event logging	Python Session State / SQLite

6. Feature-by-Feature Walkthrough

Every screen and function

Upload Plan Mode

Parse and analyze EXPLAIN ANALYZE JSON files offline without any database connection.

Live Query Mode

Connect to PostgreSQL, execute queries safely, and visualize live plan performance.

Natural Language to SQL

Convert plain English prompts to SQL using Gemini or fast offline matching.

ML Cardinality Comparison

Compare PostgreSQL estimates vs actual rows vs XGBoost predictions per node.

AI Query Doctor

Get automated risk ratings, detected anti-patterns, and actionable optimization advice.

Smart Index Advisor

Rule-based index suggestions for unindexed sequential scans.

Activity Logs & Sample Analysis

Inspect session activity and run built-in demonstration plans instantly.

7. How to Explain in an Interview

30-sec and 90-sec answers

30-Second Summary

AI Query Optimizer is a Streamlit dashboard that makes PostgreSQL execution plans understandable. It uses EXPLAIN ANALYZE JSON to get real plan data, an XGBoost model to predict node cardinality, and Q-error to compare the model against PostgreSQL estimates. Gemini provides optional plain-English advice. It is safe by design: read-only SELECT queries only, with zero auto-execution of database changes.

90-Second Summary

A slow SQL query is hard to debug because database planners make row-count estimates at nested steps. When those estimates fail, PostgreSQL picks inefficient join algorithms or table scans. My app flattens execution plan nodes and evaluates PostgreSQL estimates, actual rows, and XGBoost predictions using Q-error. The user sees exact bottlenecks visually via Plotly charts, receives rule-based index recommendations, and gets Gemini AI summaries. Security is enforced with single SELECT validation, secret management, and strictly informational advice rather than autonomous database modifications.

8. Counter Questions & Answers

Essential interview prep

Q: What is Q-error?

A symmetric ratio metric: $\max(\text{estimate}/\text{actual}, \text{actual}/\text{estimate})$. 1.0 is perfect accuracy.

Q: Why XGBoost over Neural Networks?

Tabular plan features perform better and train faster on tree-based models like XGBoost, which are also easier to explain.

Q: Does Gemini train the ML model?

No. Gemini provides language translation and text summaries. XGBoost handles numerical cardinality predictions.

Q: Why EXPLAIN ANALYZE?

EXPLAIN gives estimates only. ANALYZE runs the query to capture actual row counts needed to compute Q-error.

Q: How is SQL execution kept safe?

Regex extracts pure SQL, validates it as a single SELECT statement, and blocks DDL/DML keywords.

9. Glossary

Terms in plain English

Term	Definition
Cardinality	The number of rows returned by a query operation.
Seq Scan	Scanning every row in a table sequentially.
Index Scan	Using an index tree to jump directly to matching rows.
Hash Join	A join method that builds a hash table of one relation to match another.
Q-error	Ratio metric measuring estimate accuracy relative to actual rows.
Streamlit	Python framework for building interactive data web applications.

10. Deployment Guide

A beginner-safe guide for the live hosted application

Repository roles

The original shared repository is **Oxide06/AI-Query-Optimizer**. The personal fork is **Harshit-Kumar-1710/AI-Query-Optimizer**. A fork is your copy of a repository: make and test changes in the fork, then open a Pull Request to the original repository when the team wants to merge them.

A. Keep secrets out of GitHub

- Never paste an API key, database password, or full Neon connection string into app.py, README screenshots, commits, or chat messages.
- Keep .env, .streamlit/secrets.toml, .gemini_key.json, and *.db files ignored by Git. The project .gitignore is the safety net, but you must still check git status before pushing.
- Use Streamlit Secrets for the deployed app. Use environment variables or a local .env file only for local development.

B. Deploy to Streamlit Community Cloud - step by step

- Push the tested code to GitHub. For the shared version, use the original repository; for personal testing, use the fork.
- Open share.streamlit.io, sign in with GitHub, click Create app, and select the repository and the main branch.
- Set the main file path to app.py. Streamlit reads requirements.txt and installs the Python packages automatically.
- Open App settings, then Secrets. Paste the secret template below after replacing every placeholder. Save it and choose Reboot app.
- First test Sample Analysis and Upload Plan. These should work without any live database. Then test Fast Offline Mode. Finally test Live Query after the hosted PostgreSQL secrets are correct.
- If requirements.txt changes, redeploy or reboot so Streamlit installs the updated packages.

C. Streamlit Secrets template

```
GEMINI_API_KEY = "your_key_from_google_ai_studio"
GEMINI_MODEL = "gemini-2.5-flash"
DB_HOST = "your-neon-host.neon.tech"
DB_PORT = "5432"
DB_NAME = "neondb"
DB_USER = "your_database_user"
DB_PASSWORD = "your_database_password"
DB_SSLMODE = "require"
```

D. Create the correct Gemini key

- Open Google AI Studio and create or copy a Gemini API key for your project. Store that value as GEMINI_API_KEY in Streamlit Secrets.
- Do not use a Google browser login token, a value beginning with Bearer, or an OAuth access token. Those are different credentials and will fail with ACCESS_TOKEN_TYPE_UNSUPPORTED.
- After changing the secret, save it and reboot the Streamlit app. The sidebar should say that the Gemini key is active. Use Fast Offline Mode while testing database features without Gemini.

E. Configure Neon or another hosted PostgreSQL database

- Copy the host, database, user, password, and port from the provider dashboard. Do not use localhost in a cloud deployment because localhost means the Streamlit container itself, not your computer.
- For Neon, Supabase, Aiven, and most managed PostgreSQL providers, set DB_SSLMODE to require. The app now passes both DB_PORT and DB_SSLMODE to psycopg2.
- Create a read-only database user for the demo whenever possible. The app validates SELECT queries, but database permissions are the final protection.
- Load the dvd_rental schema/data into the hosted database if you want the built-in sample SQL queries to run. Otherwise use Upload Plan and Sample Analysis for a database-free demo.

F. Error checklist

What you see	Likely cause	Simple fix
ACCESS_TOKEN_TYPE_UNSU PPORTED	OAuth/login token was saved as GEMINI_API_KEY.	Replace it with a Gemini API key from Google AI Studio and reboot.
password authentication failed	Database user/password is wrong.	Copy both values again from the database provider and update Secrets.
SSL connection required	Managed database requires encrypted transport.	Set DB_SSLMODE = "require".
connection timeout / could not connect	Wrong host/port or database network is unavailable.	Use the provider host, correct port, then test provider status. Do not use localhost on Streamlit Cloud.
model/API error	Package or selected model is unavailable, quota is exhausted, or key lacks access.	Keep google-genai in requirements.txt, use GEMINI_MODEL = "gemini-2.5-flash", check key/quota, and reboot.
Only one read-only SELECT allowed	The input contains a write statement, multiple statements, or comments.	Submit one plain SELECT query only.

Key Point

Deployment order to remember: 1) push code, 2) deploy app, 3) add secrets, 4) reboot, 5) test offline features, 6) test Gemini, 7) test hosted database, 8) confirm Activity Logs do not expose secrets.